

maquette-scad

Parametric CAD in Typst, via OpenSCAD + Manifold

github.com/bernsteining/maquette · bernsteining.github.io/maquette



Version 0.1.0 · August 26, 2026

CONTENTS

Contents

Introduction	3
Quickstart	4
Primitives	5
3D	5
2D $\rightarrow$ 3D	6
Boolean operations	9
Transforms	11
Procedural placement (Typst for)	13
2D primitives	14
Offset	15
Compiling an existing .scad file	17
Language coverage	18
Design notes	19

Introduction

maquette-scad extends **maquette** with procedural geometry: takes an OpenSCAD-flavored expression (or a `.scad` source string), compiles it to a watertight mesh via the **Manifold** CSG kernel, and returns PLY bytes that plug straight into **maquette**'s `render-ply` — all inside Typst's `wasm` sandbox. No external OpenSCAD install, no shell-out, no intermediate `.stl` on disk.

Two entry points:

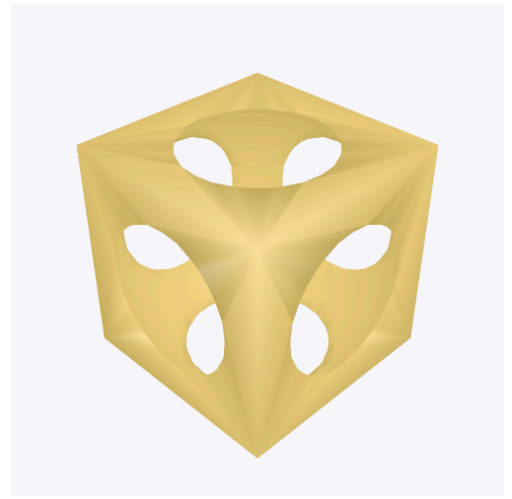
- **Typst DSL** — write geometry as Typst expressions using `cube`, `sphere`, `translate`, `difference` etc. Terse, composable, uses Typst's own `for` / `range` / `calc` for procedural placement.
- **.scad ingestion** — hand in the text of an existing `.scad` file (`read("part.scad")`). Full OpenSCAD language surface: `function`, `module`, `for`, list comprehensions, `include` / `use`, `$fn` / `$fa` / `$fs`.

The CSG kernel guarantees watertight boolean output — none of the “20% of faces open on any non-trivial cut” issue you get from BSP-based kernels.

This document only covers the **geometry** layer — primitives, booleans, transforms, `.scad` ingestion, language coverage. Once you have PLY bytes, everything downstream (camera, lighting, materials, shadows, tone mapping, ...) is **maquette**'s job — see **maquette**'s documentation for those knobs. See `crates/maquette-scad/FIDELITY.md` for the exhaustive OpenSCAD language coverage tracker.

Quickstart

```
#let part = scadypst(  
  difference(  
    cube(20, center: true),  
    sphere(12, fn: 48),  
  )  
)  
#show-part(part)
```



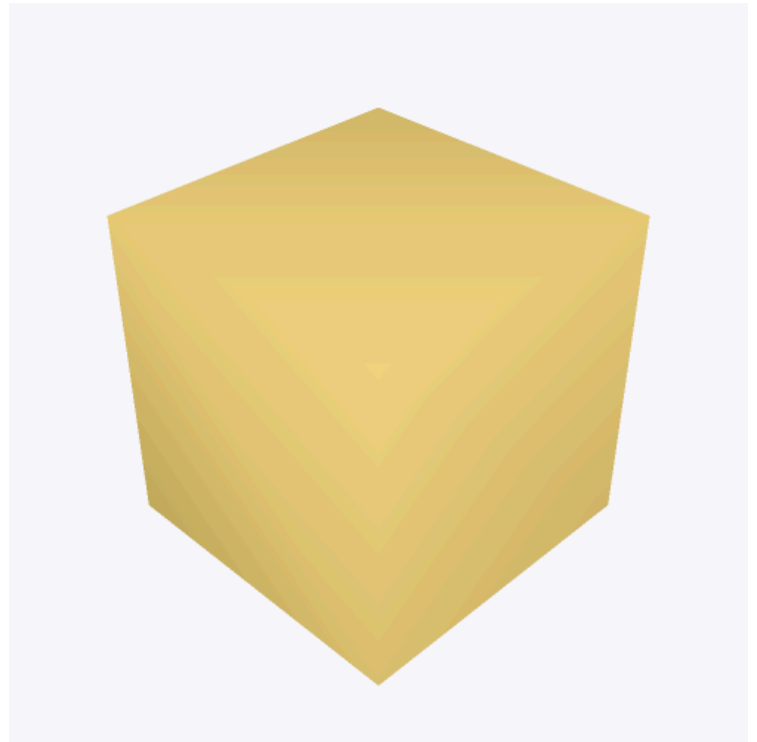
Two things to notice:

- cube / sphere / difference build an expression tree of dicts — nothing runs yet.
- scadypst(tree) sends the tree to the wasm compiler, which returns PLY bytes.
- show-part in this doc is a thin wrapper around #render-ply(bytes, ..config) — you'd inline the render config yourself in real use.

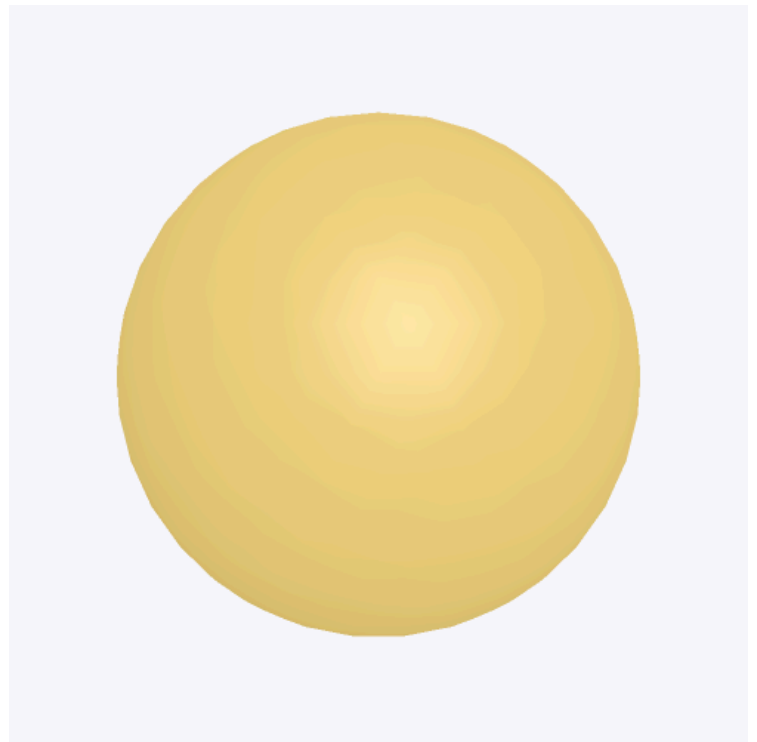
Primitives

3D

```
#show-part(scadypst(cube(20, center: true)))
```



```
#show-part(scadypst(sphere(15, fn: 48)))
```



```
#show-part(scadypst(cylinder(30, r: 10, center: true, fn: 48)))
```



Cone / frustum: pass r1 and r2 instead of r.

```
#show-part(scadypst(cylinder(30, r1: 15, r2: 5, center: true, fn: 48)))
```



2D → 3D

square / circle / polygon on their own return a 2D shape. linear-extrude and rotate-extrude lift them into 3D.

```
#show-part(scadypst(linear-extrude(20, star(5, 15, 6))))
```



linear-extrude supports a twist (degrees over the full height) and a taper scale:

```
#show-part(scadypst(linear-extrude(30, square(15, center: true),  
twist: 90, scale: 0.5)))
```



rotate-extrude revolves a 2D profile (living in the +x half-plane) around the z-axis. Classic torus:

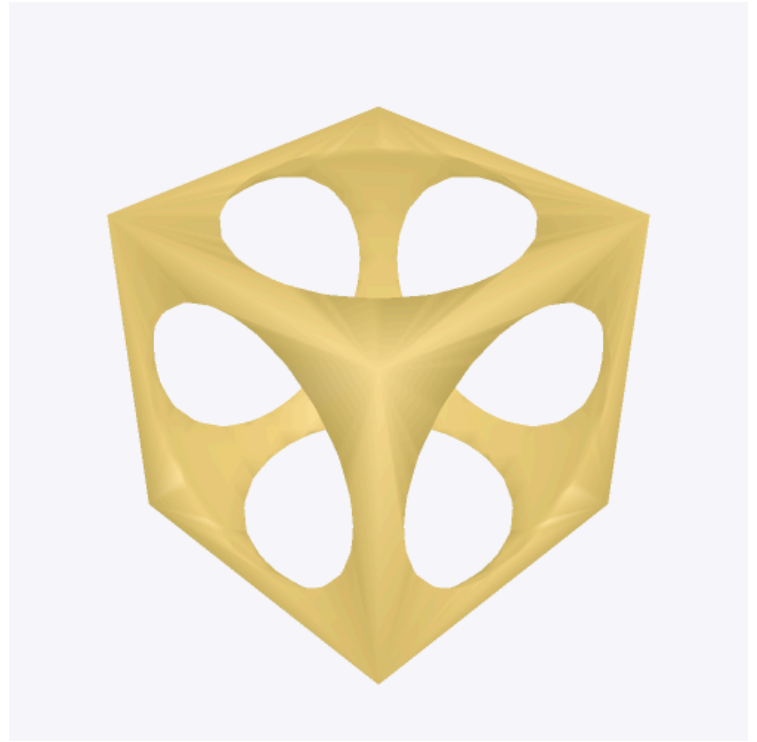
```
#show-part(scadypst(rotate-extrude(translate((15, 0, 0),  
circle(5, fn: 32)), fn: 64)))
```



Boolean operations

union, difference, intersection — variadic, take any number of children. difference is first-minus-the-rest.

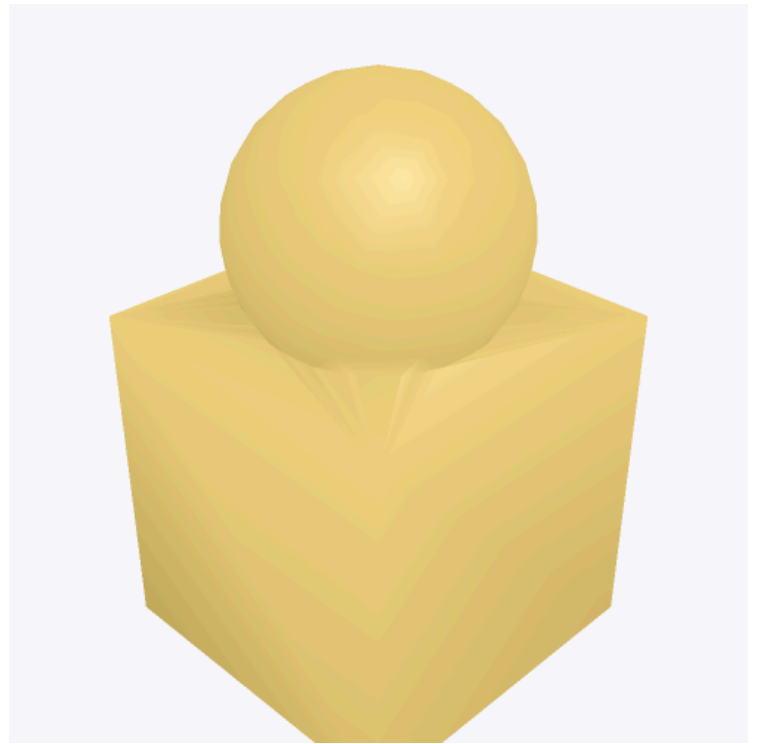
```
#show-part(scadypst(  
  difference(  
    cube(20, center: true),  
    sphere(13, fn: 48),  
  )  
))
```



```
#show-part(scadypst(  
  intersection(  
    cube(20, center: true),  
    sphere(13, fn: 48),  
  )  
))
```



```
#show-part(scadypst(
  union(
    cube(20, center: true),
    translate((0, 0, 15), sphere(8, fn: 32)),
  )
))
```



`hull` (convex hull of the union) and `minkowski` (Minkowski sum) are 3D-only.

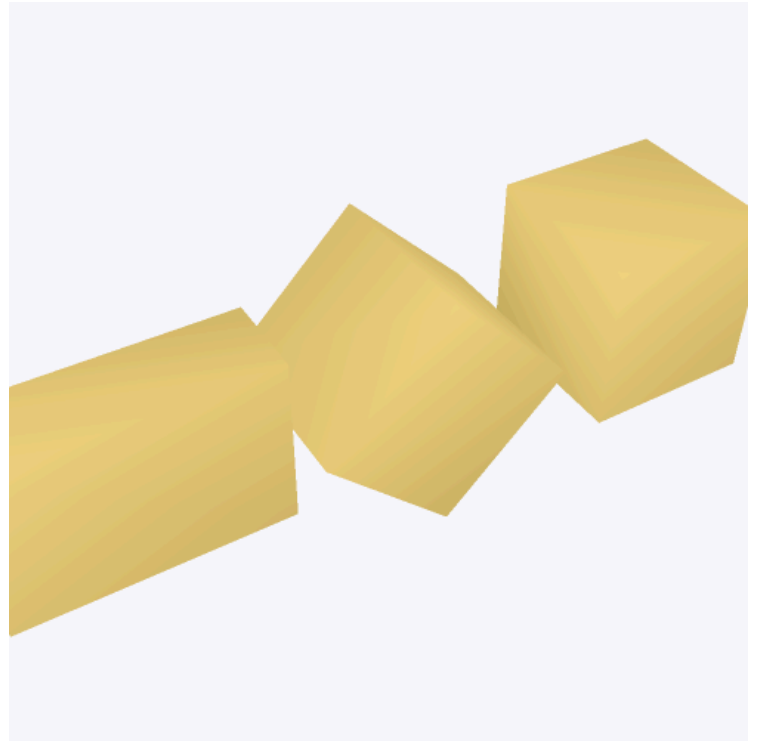
```
#show-part(scadypst(
  hull(
    translate((-15, 0, 0), sphere(6, fn: 24)),
    translate(( 15, 0, 0), sphere(6, fn: 24)),
  )
))
```



Transforms

translate / rotate / scale / mirror / resize all take a child at the end.

```
#show-part(scadypst(  
  union(  
    cube(10, center: true),  
    translate((15, 0, 0), rotate((0, 45, 0), cube(10, center:  
true))),  
    translate((30, 0, 0), scale((1.5, 0.5, 1), cube(10, center:  
true))),  
  )  
))
```



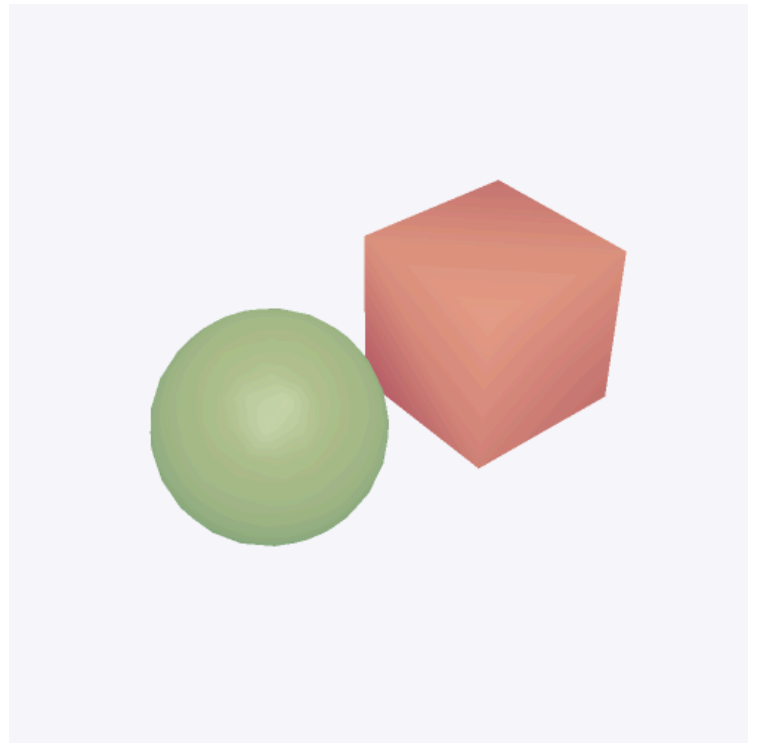
mirror reflects across a plane whose normal is the vector you pass.

```
#show-part(scadypst(  
  union(  
    translate((-8, 0, 0), cylinder(20, r1: 8, r2: 2, fn: 32)),  
    mirror((1, 0, 0), translate((-8, 0, 0), cylinder(20, r1: 8,  
r2: 2, fn: 32))),  
  )  
))
```



color propagates through boolean ops; the emitted PLY carries per-face RGB, which render-ply picks up automatically:

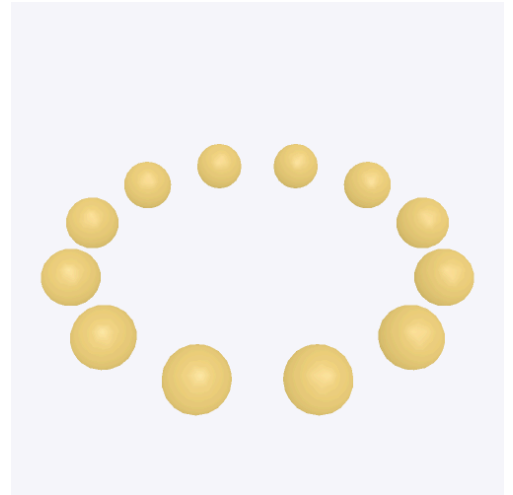
```
#show-part(  
  scadypst(union(  
    color((0.9, 0.3, 0.3), cube(10, center: true)),  
    color((0.3, 0.7, 0.4), translate((15, 0, 0), sphere(6, fn:  
32))),  
  )),  
  color: none,  
)
```



Procedural placement (Typst for)

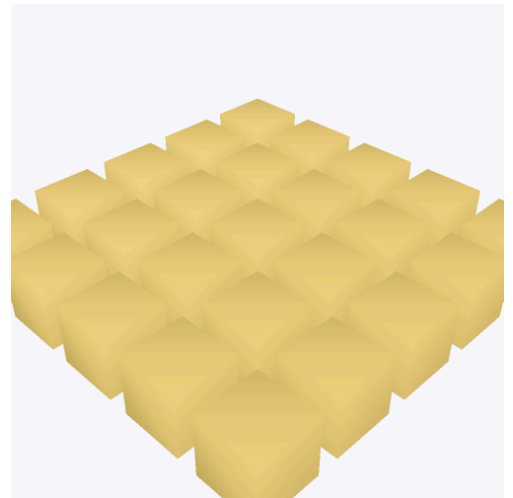
The DSL sits in Typst. You don't need OpenSCAD's `for()` — use Typst's own iteration.

```
#let ring = union(..for i in range(12) {  
  let a = i * 30  
  let r = 20  
  (translate(  
    (calc.cos(a * 1deg) * r, calc.sin(a * 1deg) * r, 0),  
    sphere(3, fn: 24),  
  ),)  
})  
#show-part(scadypst(ring))
```



Nested loops for a grid:

```
#let grid = union(..for x in range(-2, 3) {  
  for y in range(-2, 3) {  
    (translate(  
      (x * 8, y * 8, 0),  
      cube(6, center: true),  
    ),)  
  }  
})  
#show-part(scadypst(grid))
```



2D primitives

For extrusion sources. Render one by extruding it a tiny amount so we can see the outline:

```
#show-part(scadypst(linear-extrude(1, star(6, 15, 6))))
```



```
#show-part(scadypst(linear-extrude(1, ngon(6, 10))))
```

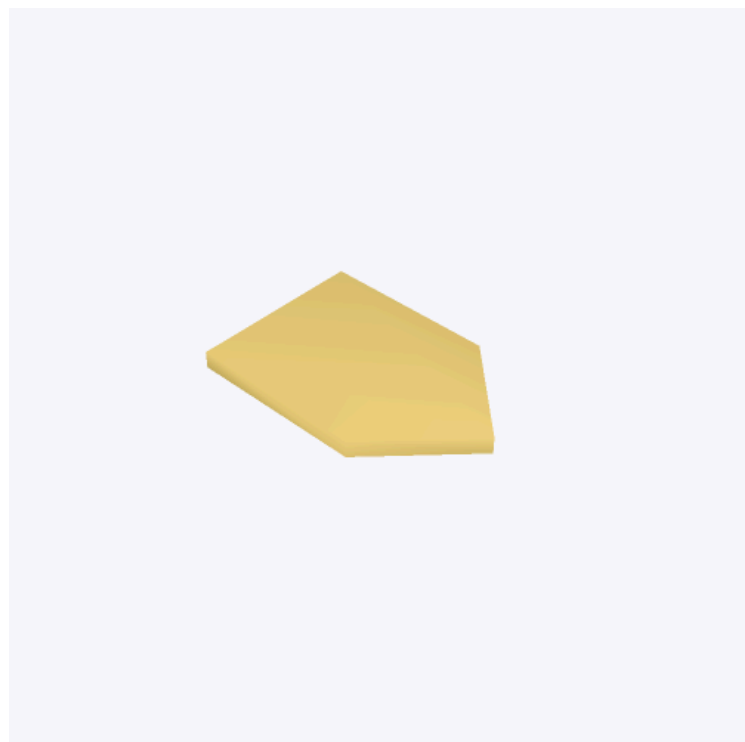


```
#show-part(scadypst(linear-extrude(1, rounded-square(20, 12, 3,
fn: 24))))
```



`polygon` — arbitrary 2D shapes from (x, y) point lists. `paths` (optional) is a list of index-rings: first ring is the outer boundary, rest are holes.

```
#show-part(scadypst(linear-extrude(1,
  polygon(((0, 0), (10, 0), (10, 10), (5, 15), (0, 10))),
)))
```



Offset

`offset(d, shape2d)` grows ($d > 0$) or shrinks ($d < 0$) a 2D shape by d units. Useful for creating outlines / clearance.

```
#show-part(scadypst(linear-extrude(1,  
    offset(2, square(10, center: true)),  
    )))
```



Compiling an existing .scad file

`compile-scad(source)` takes real OpenSCAD source (parsed via `openscad-rs`, evaluated in-crate). Full language coverage per FIDELITY.md.

```
#import "@preview/maquette-scad:0.1.0": compile-scad
#import "@preview/maquette:0.1.3": render-ply

#let part = compile-scad(read("part.scad"))
#render-ply(part)
```

For .scad files that use or include other .scad files, or that import an STL/OBJ mesh, supply the sidecar bytes through the `files` / `bin` / `font` params:

```
#compile-scad(read("main.scad"),
  files: (
    "utils.scad": read("utils.scad"),
    "gears.scad": read("gears.scad"),
  ),
  bin: (
    "mount.stl": read("mount.stl", encoding: none),
  ),
  font: read("logo.ttf", encoding: none),
  fn: 48,
)
```

The `fn` argument sets a default `$fn` for the compile (overridable per-primitive via `fn`: on cube / sphere / etc.). `trace: some-path-string` dumps the evaluator trace to the given path — useful when a nested `for` or `module` isn't producing what you expected.

Language coverage

The .scad evaluator is complete for everything a typical file uses. TL;DR:

Supported (✓) — comments, numbers, bool, string, undef, vectors, ranges, variable assignment (last-wins scope), `let()` expression, member access, indexing, all standard operators, `$fn` / `$t` / `$preview` / `$children`, all 2D + 3D primitives, all transforms (translate, rotate, scale, mirror, resize, multmatrix, color, offset), all booleans (union, difference, intersection), extrusions (`linear_extrude` with `twist` / `scale` / `slices`, `rotate_extrude`), hull, minkowski, projection, list comprehensions, `for`, `if`, `intersection_for`, `function`, `module`, `render`, string ops, math functions, `include` + `use`, `color()` with per-face RGB propagation.

Partial (⊗) — `$fa` / `$fs` are defined so libraries reading them work, but our primitive builder tessellates from `$fn` (or `fn`: on the primitive), not adaptively.

Not supported (✗) — `import()` of external STL/OBJ/DXF (wasm sandbox has no filesystem — pass bytes via `bin`: instead), `surface()` heightmap import, `$vpr` / `$vpt` / `$vpd` / `$vpf` viewport variables (n/a — no viewport at eval time), font-dependent text glyphs beyond basic Latin-1 skeletons.

See `crates/maquette-scad/FIDELITY.md` for the exhaustive per-feature tracker.

Design notes

Why Manifold? — the previous kernel we used (csgrs 0.20.1, BSP-based) left about 20% of faces open on any non-trivial boolean (bores, notches, partial overlaps). Manifold guarantees watertight, correctly-triangulated output. Every mesh this crate emits has zero open edges, verified on the full test corpus.

Why compile in-crate? — a .scad-then-shell-out design would need a filesystem, a working OpenSCAD binary on the host, and IPC across a process boundary. The wasm plugin has none of that. Ingesting .scad text ourselves keeps the whole compile deterministic under Typst’s sandbox — same input, same PDF, cross-platform.

Two wasm modules. — `maquette-scad.wasm` (this crate) compiles geometry to PLY. `maquette.wasm` (the sibling plugin) renders the PLY. They only exchange the PLY blob; you can also send `scadypst(...)`’s output to `maquette-gltf` via a .gltf wrapper if you want PBR shading on procedural geometry.